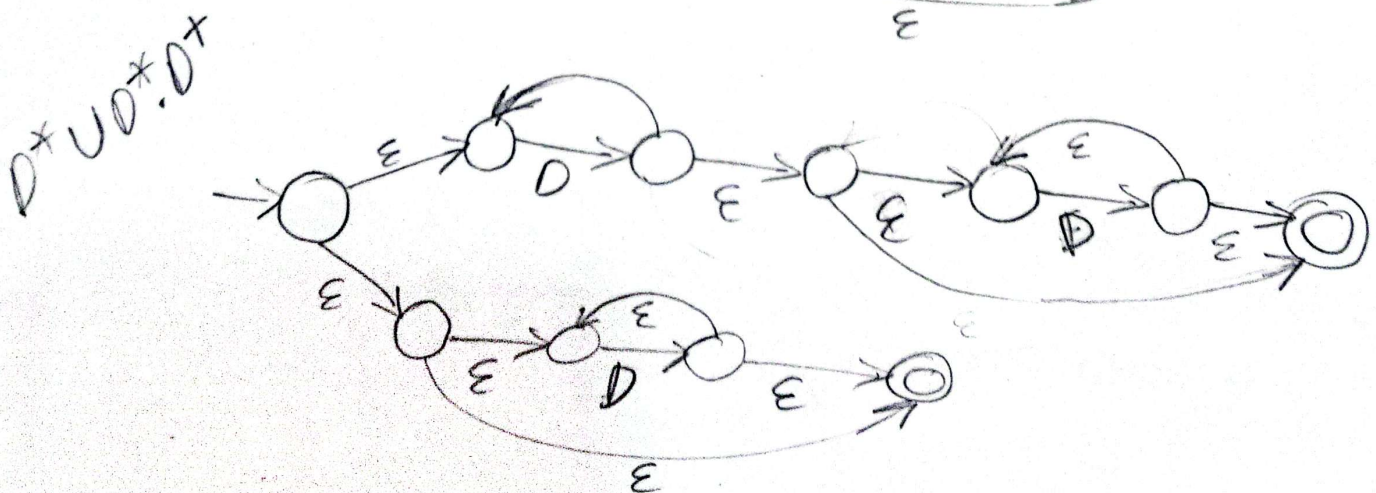
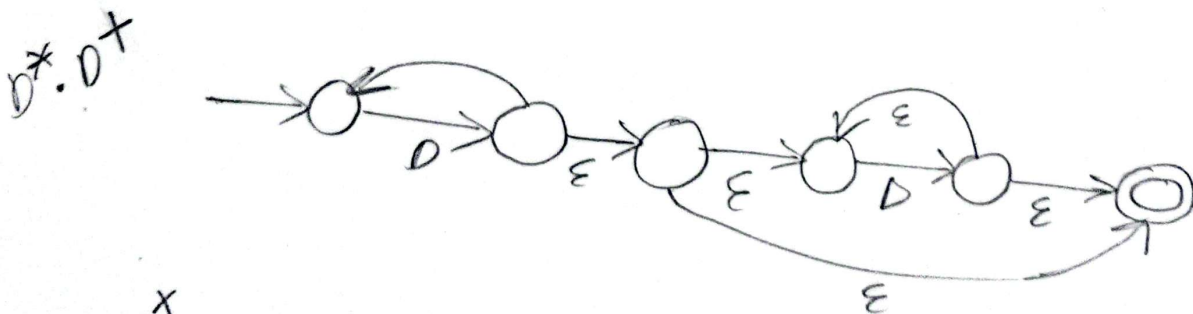
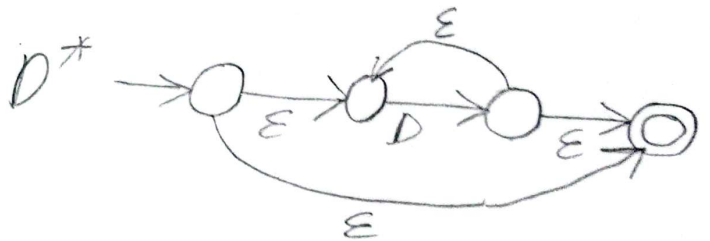


$$(+U - U\varepsilon)(D^+ U D^+ . D^+ U D^+ . D^+)$$

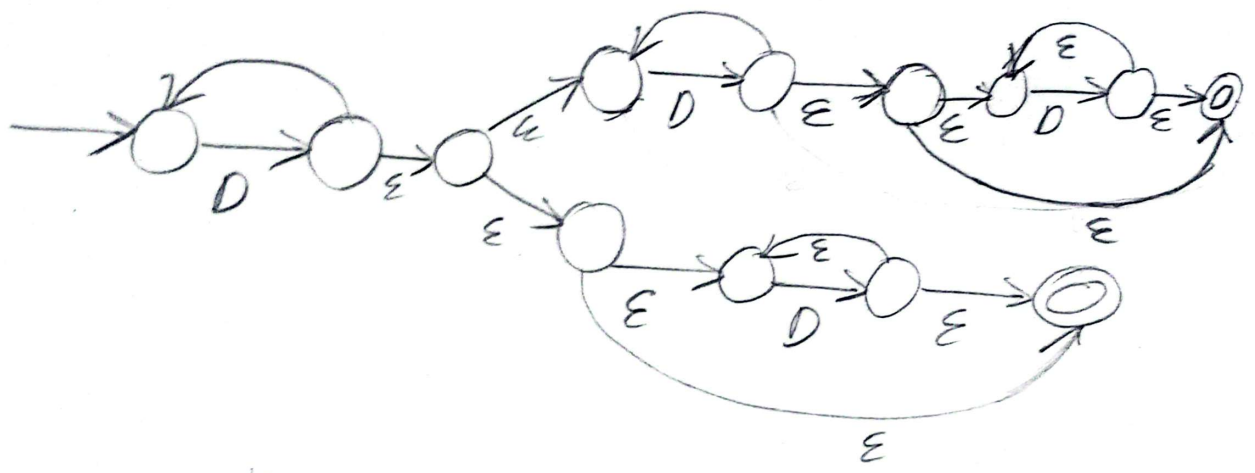
where $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Converting to NFA: $D \rightarrow$ 

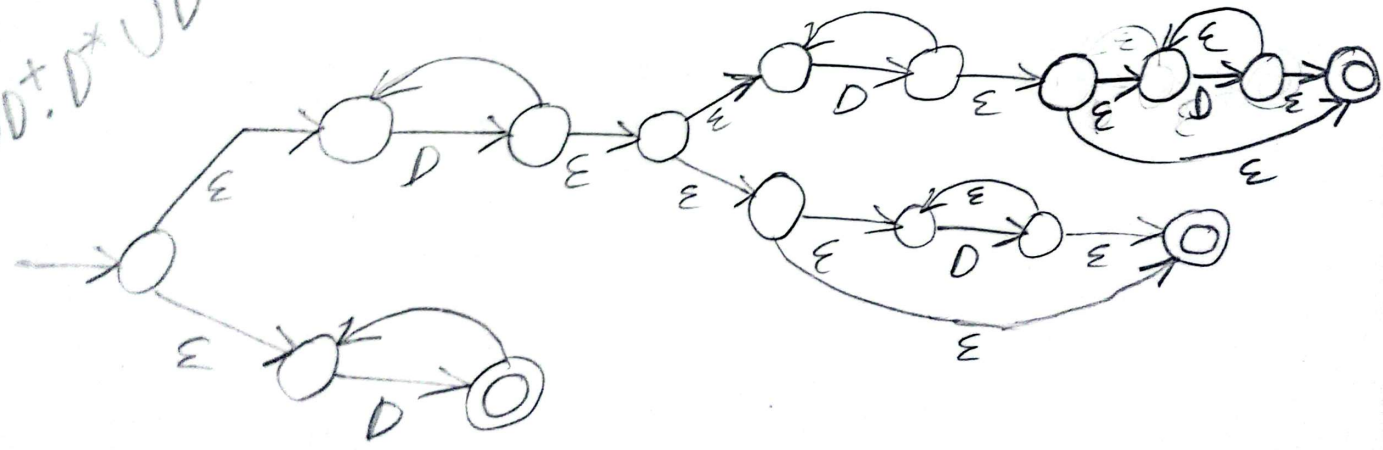


(2)

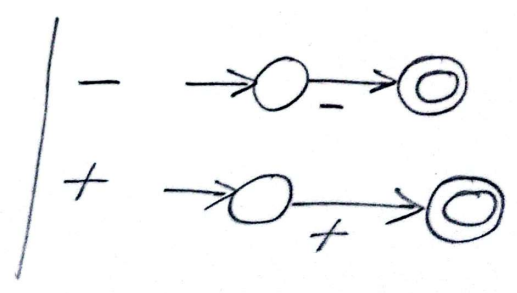
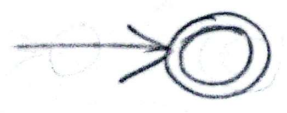
$D^+ . D^* U D^+ . D^+$



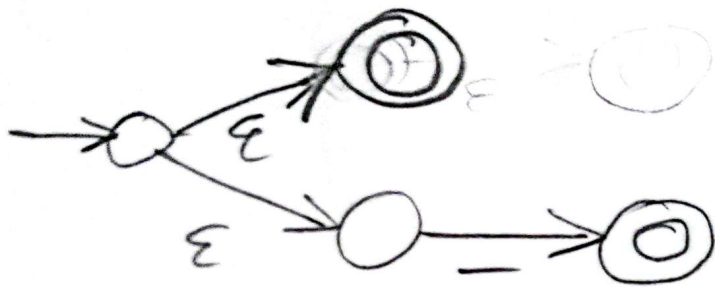
$D^+ U D^+ . D^* U D^+ . D^+$



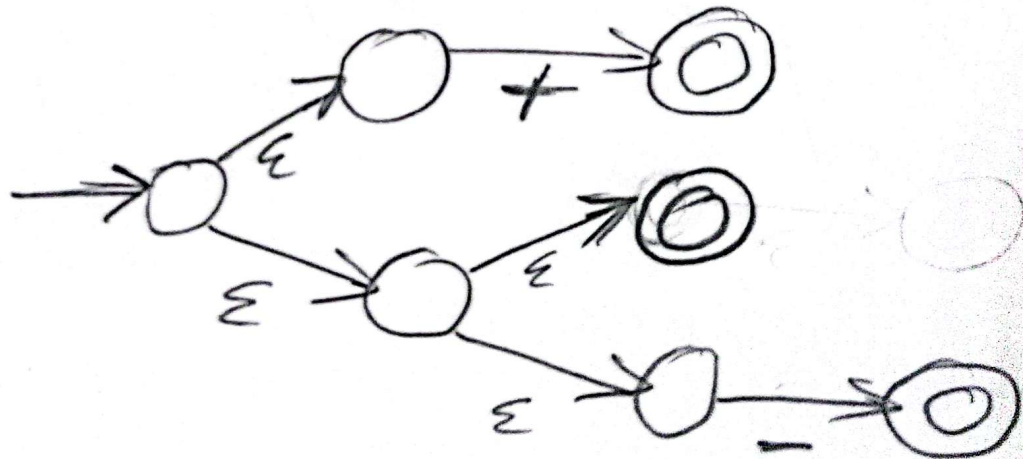
ϵ



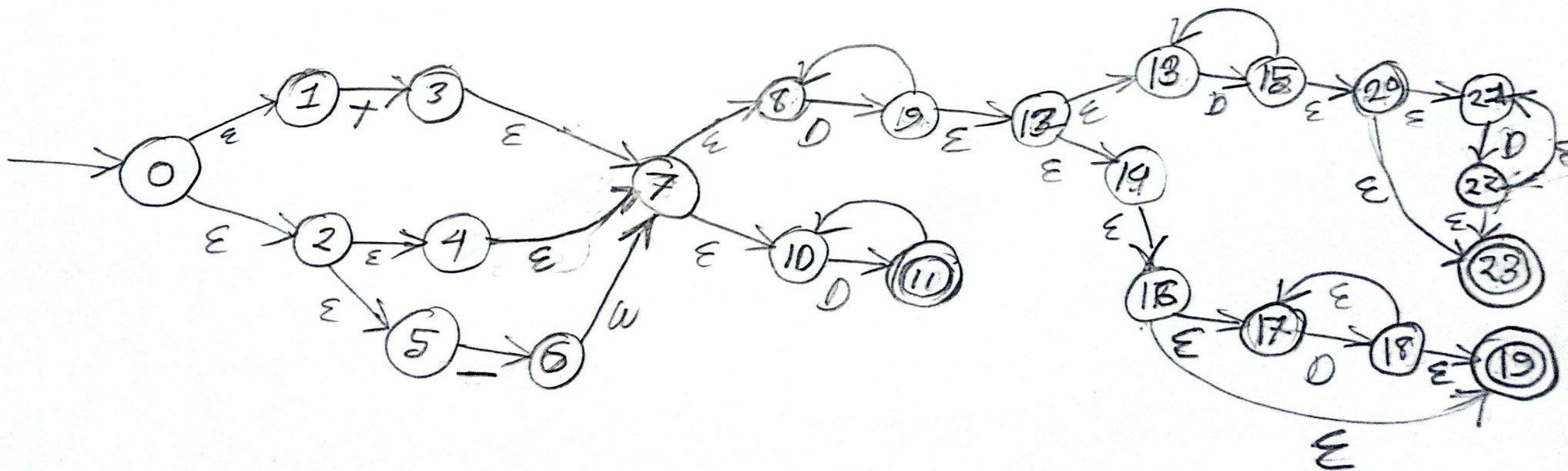
$-U\epsilon$



$+U-U\epsilon$



$(+U-UE)(D+UD^+.D^*UD^*.D^+)$

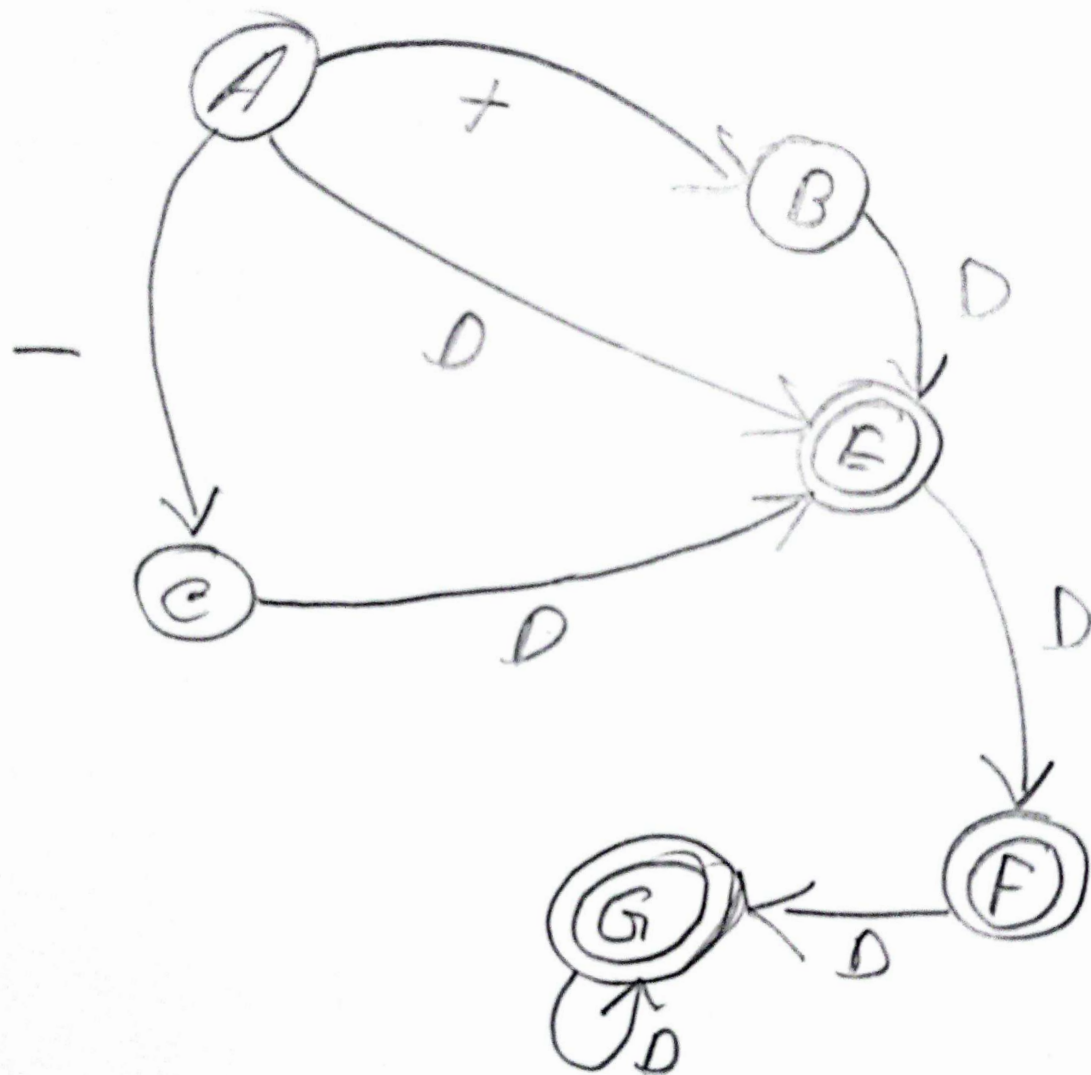


Part (b) NFA to equivalent DFA

DFA	ϵ -closure of	ϵ -closure outcomes
A	ϵ -closure($\{0\}$)	0, 1, 2, 4, 5, 7, 8, 10
B	ϵ -closure($\{3\}$)	3, 7, 8, 10
C	ϵ -closure($\{6\}$)	6, 7, 8, 10
E	ϵ -closure($\{9, 11\}$)	9, 11, 12, 13, 14, 16, 17
F	ϵ -closure($\{15, 18\}$)	15, 18, 17, 19, 20, 23, 21
G	ϵ -closure($\{22, 16\}$)	17, 18, 19, 21, 22, 23

DFA states	+	-	D
A	B	C	E
B	$\emptyset$	$\emptyset$	E
C	$\emptyset$	$\emptyset$	E
E	$\emptyset$	$\emptyset$	F
F	$\emptyset$	$\emptyset$	G
G	$\emptyset$	$\emptyset$	G

DFA:



Part C

Converting DFA from part (b) to equivalent C++ program.

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{ char dfa-current-state;
```

```
int s;
```

```
cout << "Number of accept states:";
```

```
cin >> s;
```

```
char dfa-accept-states[s];
```

```
cout << "Enter accept states:";
```

```
for (int i = 0; i < s; i++)
```

```
{ cin >> dfa-accept-state[i]; }
```

```
char dfa-current-input;
```

```
cout << "current state: ";
```

```
cin >> dfa_current_state;
```

```
cout << "Enter sequence of input. end with '.'" << endl;
```

```
cin >> dfa_current_input;
```

```
while (dfa_current_input != '.') {
```

```
    if (dfa_current_state == 'A' && (dfa_current_input >= '0' && dfa_current_input <= '9'))
```

```
    { dfa_current_state = 'E'; }
```

```
    else if (dfa_current_state == 'A' && dfa_current_input == '+')
```

```
    { dfa_current_state = 'B'; }
```

```
    else if (dfa_current_state == 'A' && dfa_current_input == '-')
```

```
    { dfa_current_state = 'C'; }
```

else if (dfa_current_state = 'B' && (dfa_current_input
>= '0' && dfa_current_input <= '9'))

{ dfa_current_state = 'E'; }

else if (dfa_current_state = 'C' && (dfa_current_input
>= '0' && dfa_current_input <= '9'))

{ dfa_current_state = 'E'; }

else if (dfa_current_state = 'E' &&
(dfa_current_input >= '0' &&
dfa_current_input <= '9'))

{ dfa_current_state = 'F'; }

else if (dfa_current_state = 'f' &&
(dfa_current_input >= '0' && dfa_current_input <= '9'))

{ dfa_current_state = 'G'; }

```
else if (dfa_current_state == 'G' &&  
        (dfa_current_input == '0' && dfa_current_input != '9'))
```

```
{ dfa_current_state = 'G'; }
```

```
else { cout << "invalid input" << endl;  
        return 0; }
```

```
cout << "current-state;" << dfa_current_state <<  
        endl;
```

```
cin >> dfa_current_input;
```

```
bool is_valid = false;
```

```
for (int i = 0; i < 5; i++)
```

```
{ if (dfa_current_state == dfa_accept_state[i])
```

```
{ is_valid = true;
```

```
    break;
```

```
}
```

```
if (is_valid) =
```

```
{ cout << "valid lexeme" << endl; }
```

```
else { cout << "invalid lexeme" << endl; }
```

```
return 0;
```

```
}
```